US Traffic Safety Analysis Notebook

Overview

> This notebook analyzes the US Accidents dataset (2016-2023) containing approximately 7.7 million traffic accident records across 49 US states. The data was collected through various traffic APIs and provides insights into accident patterns, hotspots, and contributing factors.

Dataset Attribution

Dataset provided by Sobhan Moosavi et al. under CC BY-NC-SA 4.0 license.
Citations:

- Moosavi et al. "A Countrywide Traffic Accident Dataset.", 2019
- Moosavi et al. "Accident Risk Prediction based on Heterogeneous Sparse Data: New Dataset and Insights." ACM SIGSPATIAL, 2019

Research Objectives

1. Identify accident hotspots and temporal patterns
2. Analyze environmental factors contributing to accidents
3. Study impact of COVID-19 on traffic patterns
4. Develop predictive insights for accident prevention

Setup and Dependencies

```
Required libraries: - pandas - numpy - matplotlib - seaborn - kagglehub - folium (for geospatial visualization)
```

Data Dictionary Preview

Key features include:

- **ID:** Unique identifier
- **Severity:** Accident impact level (1-4)
- **Start_Time/End_Time:** Temporal data
- **Lat/Lng:** Geographic coordinates
- **Description:** Accident details
- **Environmental conditions:** Weather, visibility etc.

In [ ]:
```python
# Import required libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import kagglehub
import folium

# Download dataset
path = kagglehub.dataset_download("sobhanmoosavi/us-accidents")
print("Path to dataset files:", path)
```

Data Quality Assessment

Objectives:

> 1. Load and examine data structure 2. Check for missing values and anomalies 3. Identify duplicate records 4. Validate data types and value ranges 5. Document data quality findings

Key metrics to assess:

- Row and column counts - Memory usage - Missing value percentages - Data type consistency - Value distributions - Duplicate records - Invalid/outlier values

In [5]:
```
# Load the dataset
df = pd.read_csv(f"{path}/US_Accidents_March23.csv")

# Basic information about the dataset
print("Dataset Shape:", df.shape)
print("\nMemory Usage:", df.memory_usage().sum() / 1024**2, "MB")
print("\nData Types:")
print(df.dtypes)

# Missing values analysis
missing_values = df.isnull().sum()
missing_percentages = (missing_values / len(df)) * 100
print("\nMissing Values Analysis:")
print(pd.DataFrame({
    'Missing Values': missing_values,
    'Percentage': missing_percentages
}).query('`Missing Values` > 0'))

# Duplicate check
duplicates = df.duplicated().sum()
print(f"\nNumber of duplicate rows: {duplicates}")

# Basic statistics for numeric columns
print("\nNumeric Columns Statistics:")
print(df.describe())

# Value counts for categorical columns
for col in df.select_dtypes(include=['object']).columns:
    print(f"\nUnique values in {col}:")
    print(df[col].value_counts().head())
```

```
Dataset Shape: (7728394, 46)

Memory Usage: 2041.592851638794 MB

Data Types:
ID                      object
Source                  object
Severity                 int64
Start_Time              object
End_Time                object
Start_Lat              float64
Start_Lng              float64
End_Lat                float64
End_Lng                float64
Distance(mi)           float64
Description             object
Street                  object
City                    object
County                  object
State                   object
Zipcode                 object
Country                 object
Timezone                object
Airport_Code            object
Weather_Timestamp       object
Temperature(F)         float64
Wind_Chill(F)          float64
Humidity(%)            float64
Pressure(in)           float64
Visibility(mi)         float64
Wind_Direction          object
```

```
Wind_Speed(mph)          float64
Precipitation(in)        float64
Weather_Condition         object
Amenity                     bool
Bump                        bool
Crossing                    bool
Give_Way                    bool
Junction                    bool
No_Exit                     bool
Railway                     bool
Roundabout                  bool
Station                     bool
Stop                        bool
Traffic_Calming             bool
Traffic_Signal              bool
Turning_Loop                bool
Sunrise_Sunset            object
Civil_Twilight            object
Nautical_Twilight         object
Astronomical_Twilight     object
dtype: object

Missing Values Analysis:
                       Missing Values  Percentage
End_Lat                       3402762   44.029355
End_Lng                       3402762   44.029355
Description                         5    0.000065
Street                          10869    0.140637
City                              253    0.003274
Zipcode                          1915    0.024779
Timezone                         7808    0.101030
Airport_Code                    22635    0.292881
Weather_Timestamp              120228    1.555666
Temperature(F)                 163853    2.120143
Wind_Chill(F)                 1999019   25.865904
Humidity(%)                    174144    2.253301
Pressure(in)                   140679    1.820288
Visibility(mi)                 177098    2.291524
Wind_Direction                 175206    2.267043
Wind_Speed(mph)                571233    7.391355
Precipitation(in)             2203586   28.512858
Weather_Condition              173459    2.244438
Sunrise_Sunset                 23246    0.300787
Civil_Twilight                 23246    0.300787
Nautical_Twilight              23246    0.300787
Astronomical_Twilight          23246    0.300787

Number of duplicate rows: 0

Numeric Columns Statistics:
           Severity     Start_Lat     Start_Lng       End_Lat       End_Lng  \
count  7.728394e+06  7.728394e+06  7.728394e+06  4.325632e+06  4.325632e+06
mean   2.212384e+00  3.620119e+01 -9.470255e+01  3.626183e+01 -9.572557e+01
std    4.875313e-01  5.076079e+00  1.739176e+01  5.272905e+00  1.810793e+01
min    1.000000e+00  2.455480e+01 -1.246238e+02  2.456601e+01 -1.245457e+02
25%    2.000000e+00  3.339963e+01 -1.172194e+02  3.346207e+01 -1.177543e+02
50%    2.000000e+00  3.582397e+01 -8.776662e+01  3.618349e+01 -8.802789e+01
75%    2.000000e+00  4.008496e+01 -8.035368e+01  4.017892e+01 -8.024709e+01
max    4.000000e+00  4.900220e+01 -6.711317e+01  4.907500e+01 -6.710924e+01

       Distance(mi)  Temperature(F)  Wind_Chill(F)   Humidity(%)  \
count  7.728394e+06    7.564541e+06   5.729375e+06  7.554250e+06
mean   5.618423e-01    6.166329e+01   5.825105e+01  6.483104e+01
std    1.776811e+00    1.901365e+01   2.238983e+01  2.282097e+01
min    0.000000e+00   -8.900000e+01  -8.900000e+01  1.000000e+00
25%    0.000000e+00    4.900000e+01   4.300000e+01  4.800000e+01
50%    3.000000e-02    6.400000e+01   6.200000e+01  6.700000e+01
75%    4.640000e-01    7.600000e+01   7.500000e+01  8.400000e+01
max    4.417500e+02    2.070000e+02   2.070000e+02  1.000000e+02

       Pressure(in)  Visibility(mi)  Wind_Speed(mph)  Precipitation(in)
count  7.587715e+06    7.551296e+06     7.157161e+06       5.524808e+06
mean   2.953899e+01    9.090376e+00     7.685490e+00       8.407210e-03
std    1.006190e+00    2.688316e+00     5.424983e+00       1.102246e-01
min    0.000000e+00    0.000000e+00     0.000000e+00       0.000000e+00
25%    2.937000e+01    1.000000e+01     4.600000e+00       0.000000e+00
50%    2.986000e+01    1.000000e+01     7.000000e+00       0.000000e+00
75%    3.003000e+01    1.000000e+01     1.040000e+01       0.000000e+00
max    5.863000e+01    1.400000e+02     1.087000e+03       3.647000e+01

Unique values in ID:
ID
A-7777761    1
```

```
A-1          1
A-2          1
A-3          1
A-4          1
Name: count, dtype: int64

Unique values in Source:
Source
Source1    4325632
Source2    3305373
Source3      97389
Name: count, dtype: int64

Unique values in Start_Time:
Start_Time
2021-01-26 16:16:13    225
2021-01-26 16:17:33    156
2021-02-16 06:42:43    139
2021-11-21 18:37:51    108
2020-12-16 13:53:25     96
Name: count, dtype: int64

Unique values in End_Time:
End_Time
2021-11-22 08:00:00    112
2017-05-15 15:22:55     73
2019-10-26 09:14:51     49
2018-11-25 02:51:02     46
2020-02-14 00:00:00     46
Name: count, dtype: int64

Unique values in Description:
Description
A crash has occurred causing no to minimum delays. Use caution.                                  9593
Accident                                                                                         6616
An unconfirmed report of a crash has been received. Use caution.                                 3799
A crash has occurred use caution.                                                                2894
A crash has occurred with minimal delay to traffic. Prepare to slow or move over for worker safety.    2672
Name: count, dtype: int64

Unique values in Street:
Street
I-95 N    78430
I-95 S    74528
I-5 N     71968
I-10 E    55572
I-10 W    53725
Name: count, dtype: int64

Unique values in City:
City
Miami          186917
Houston        169609
Los Angeles    156491
Charlotte      138652
Dallas         130939
Name: count, dtype: int64

Unique values in County:
County
Los Angeles    526851
Miami-Dade     251601
Orange         241275
Harris         181196
Dallas         157024
Name: count, dtype: int64

Unique values in State:
State
CA    1741433
FL     880192
TX     582837
SC     382557
NY     347960
Name: count, dtype: int64

Unique values in Zipcode:
Zipcode
91761    11247
91706    10022
92407     8922
92507     8850
33186     8375
```

```
Name: count, dtype: int64

Unique values in Country:
Country
US    7728394
Name: count, dtype: int64

Unique values in Timezone:
Timezone
US/Eastern     3580167
US/Pacific     2062984
US/Central     1645616
US/Mountain     431819
Name: count, dtype: int64

Unique values in Airport_Code:
Airport_Code
KCQT    118332
KRDU    107267
KMCJ    101786
KBNA     98926
KCLT     97273
Name: count, dtype: int64

Unique values in Weather_Timestamp:
Weather_Timestamp
2022-03-13 01:53:00    1311
2021-01-26 15:53:00     651
2022-05-13 16:53:00     558
2022-03-13 01:55:00     545
2021-01-15 22:53:00     521
Name: count, dtype: int64

Unique values in Wind_Direction:
Wind_Direction
CALM    961624
S       419989
SSW     384840
W       383913
WNW     378781
Name: count, dtype: int64

Unique values in Weather_Condition:
Weather_Condition
Fair             2560802
Mostly Cloudy    1016195
Cloudy            817082
Clear             808743
Partly Cloudy     698972
Name: count, dtype: int64

Unique values in Sunrise_Sunset:
Sunrise_Sunset
Day      5334553
Night    2370595
Name: count, dtype: int64

Unique values in Civil_Twilight:
Civil_Twilight
Day      5695619
Night    2009529
Name: count, dtype: int64

Unique values in Nautical_Twilight:
Nautical_Twilight
Day      6076156
Night    1628992
Name: count, dtype: int64

Unique values in Astronomical_Twilight:
Astronomical_Twilight
Day      6377548
Night    1327600
Name: count, dtype: int64
```

Data Quality Findings & Cleaning Plan

Key Issues Identified:

1. **Missing Values:**

   > - End coordinates (End_Lat/End_Lng): ~44% missing - Precipitation data: ~29% missing - Wind_Chill: ~26% missing - Other weather metrics: 2-7% missing

2. **Data Types:**

   > - Temporal columns (Start_Time, End_Time, Weather_Timestamp) need conversion to datetime - Boolean columns for infrastructure features (Amenity, Bump, etc.) - Numeric columns for measurements

3. **Memory Usage:**

   > - Current usage: ~2GB - Potential for optimization

Cleaning Steps:

1. **Convert data types** 2. **Handle missing values** 3. **Create derived features** 4. **Optimize memory usage**

In [7]:

```python
# Data cleaning and preprocessing
def clean_and_preprocess_data(df):
    """Clean and preprocess the accidents dataset."""

    # Create a copy to avoid modifying original data
    df_clean = df.copy()

    # 1. Convert temporal columns to datetime with ISO8601 format
    datetime_columns = ['Start_Time', 'End_Time', 'Weather_Timestamp']
    for col in datetime_columns:
        df_clean[col] = pd.to_datetime(df_clean[col], format='ISO8601')

    # Rest of the function remains the same
    # 2. Handle missing values
    weather_cols = ['Temperature(F)', 'Wind_Chill(F)', 'Humidity(%)',
                    'Pressure(in)', 'Visibility(mi)', 'Wind_Speed(mph)']
    df_clean[weather_cols] = df_clean[weather_cols].fillna(method='ffill')

    # Drop rows where critical information is missing
    critical_cols = ['Start_Lat', 'Start_Lng', 'City', 'State']
    df_clean = df_clean.dropna(subset=critical_cols)

    # 3. Create derived features
    df_clean['Duration'] = (df_clean['End_Time'] - df_clean['Start_Time']).dt.total_seconds() / 3600
    df_clean['Year'] = df_clean['Start_Time'].dt.year
    df_clean['Month'] = df_clean['Start_Time'].dt.month
    df_clean['Hour'] = df_clean['Start_Time'].dt.hour
    df_clean['DayOfWeek'] = df_clean['Start_Time'].dt.dayofweek

    # 4. Optimize memory usage
    # Convert boolean columns to bool type
    bool_columns = ['Amenity', 'Bump', 'Crossing', 'Give_Way', 'Junction',
                    'No_Exit', 'Railway', 'Roundabout', 'Station', 'Stop',
                    'Traffic_Calming', 'Traffic_Signal', 'Turning_Loop']
    df_clean[bool_columns] = df_clean[bool_columns].astype(bool)

    # Convert categorical columns to category type
    cat_columns = ['State', 'Country', 'Timezone', 'Wind_Direction',
                   'Weather_Condition', 'Sunrise_Sunset']
    for col in cat_columns:
        df_clean[col] = df_clean[col].astype('category')

    return df_clean

# Apply cleaning and preprocessing
df_clean = clean_and_preprocess_data(df)

# Print memory usage comparison
print("Original dataset memory usage:", df.memory_usage().sum() / 1024**2, "MB")
print("Cleaned dataset memory usage:", df_clean.memory_usage().sum() / 1024**2, "MB")

# Display basic info about cleaned dataset
print("\nCleaned dataset info:")
print(df_clean.info())
```

```
/var/folders/cv/87ynnq496l957k4sxtc2dshj99rmkw/T/ipykernel_15373/1565304501.py:17: FutureWarning: DataFrame.fillna with 'method' is deprecated and will raise in a future version. Use obj.ffill() or obj.bfill() instead.
  df_clean[weather_cols] = df_clean[weather_cols].fillna(method='ffill')
Original dataset memory usage: 2041.592851638794 MB
Cleaned dataset memory usage: 1975.202335357666 MB

Cleaned dataset info:
<class 'pandas.core.frame.DataFrame'>
Index: 7728141 entries, 0 to 7728393
Data columns (total 51 columns):
 #   Column                 Dtype
---  ------                 -----
 0   ID                     object
 1   Source                 object
 2   Severity               int64
 3   Start_Time             datetime64[ns]
 4   End_Time               datetime64[ns]
 5   Start_Lat              float64
 6   Start_Lng              float64
 7   End_Lat                float64
 8   End_Lng                float64
 9   Distance(mi)           float64
 10  Description            object
 11  Street                 object
 12  City                   object
 13  County                 object
 14  State                  category
 15  Zipcode                object
 16  Country                category
 17  Timezone               category
 18  Airport_Code           object
 19  Weather_Timestamp      datetime64[ns]
 20  Temperature(F)         float64
 21  Wind_Chill(F)          float64
 22  Humidity(%)            float64
 23  Pressure(in)           float64
 24  Visibility(mi)         float64
 25  Wind_Direction         category
 26  Wind_Speed(mph)        float64
 27  Precipitation(in)      float64
 28  Weather_Condition      category
 29  Amenity                bool
 30  Bump                   bool
 31  Crossing               bool
 32  Give_Way               bool
 33  Junction               bool
 34  No_Exit                bool
 35  Railway                bool
 36  Roundabout             bool
 37  Station                bool
 38  Stop                   bool
 39  Traffic_Calming        bool
 40  Traffic_Signal         bool
 41  Turning_Loop           bool
 42  Sunrise_Sunset         category
 43  Civil_Twilight         object
 44  Nautical_Twilight      object
 45  Astronomical_Twilight  object
 46  Duration               float64
 47  Year                   int32
 48  Month                  int32
 49  Hour                   int32
 50  DayOfWeek              int32
dtypes: bool(13), category(6), datetime64[ns](3), float64(13), int32(4), int64(1), object(11)
memory usage: 1.9+ GB
None
```

Data Cleaning Results

The data cleaning process was successful, with the following key outcomes:

Memory Optimization

> - Original dataset: 2,041.59 MB - Cleaned dataset: 1,975.20 MB - Memory reduction: ~66.39 MB (3.25%)

Data Structure Changes

> - Total records: 7,728,141 (253 rows removed) - 51 columns (including 5 new derived features) - Optimized data types:

- 13 boolean columns - 6 categorical columns - 3 datetime columns - 13 float columns - 4 int32 columns - 11 object columns

New Features Added

1. Duration (hours) 2. Year 3. Month 4. Hour 5. DayOfWeek

Next Steps

1. **Exploratory Data Analysis (EDA)** 2. **Geographic visualization of accident hotspots** 3. **Temporal pattern analysis** 4. **Weather impact analysis**

Exploratory Data Analysis (EDA)

Analysis Areas:

1. **Temporal Patterns**
   - Accidents by year, month, day of week, hour - Seasonal trends - Peak accident times
2. **Geographic Distribution**
   - State-level analysis - City-level hotspots - Rural vs urban patterns
3. **Severity Analysis**
   - Severity distribution - Factors affecting severity - Correlation with environmental conditions
4. **Weather Impact**
   - Weather conditions frequency - Temperature effects - Visibility impact - Precipitation correlation

In [9]:
```
# Set up visualization style
plt.style.use('seaborn-v0_8')  # Updated style name
sns.set_theme()  # Use seaborn's default theme

# Create figure for temporal analysis
fig, ((ax1, ax2), (ax3, ax4)) = plt.subplots(2, 2, figsize=(20, 15))

# 1. Accidents by Year
yearly = df_clean['Year'].value_counts().sort_index()
sns.barplot(x=yearly.index, y=yearly.values, ax=ax1)
ax1.set_title('Accidents by Year', fontsize=14)
ax1.set_xlabel('Year', fontsize=12)
ax1.set_ylabel('Number of Accidents', fontsize=12)

# 2. Accidents by Month
monthly = df_clean['Month'].value_counts().sort_index()
sns.barplot(x=monthly.index, y=monthly.values, ax=ax2)
ax2.set_title('Accidents by Month', fontsize=14)
ax2.set_xlabel('Month', fontsize=12)
ax2.set_ylabel('Number of Accidents', fontsize=12)

# 3. Accidents by Day of Week
days = ['Monday', 'Tuesday', 'Wednesday', 'Thursday', 'Friday', 'Saturday', 'Sunday']
daily = df_clean['DayOfWeek'].value_counts().sort_index()
sns.barplot(x=days, y=daily.values, ax=ax3)
ax3.set_title('Accidents by Day of Week', fontsize=14)
ax3.set_xlabel('Day', fontsize=12)
ax3.set_ylabel('Number of Accidents', fontsize=12)
ax3.tick_params(axis='x', rotation=45)

# 4. Accidents by Hour
hourly = df_clean['Hour'].value_counts().sort_index()
sns.barplot(x=hourly.index, y=hourly.values, ax=ax4)
ax4.set_title('Accidents by Hour', fontsize=14)
```

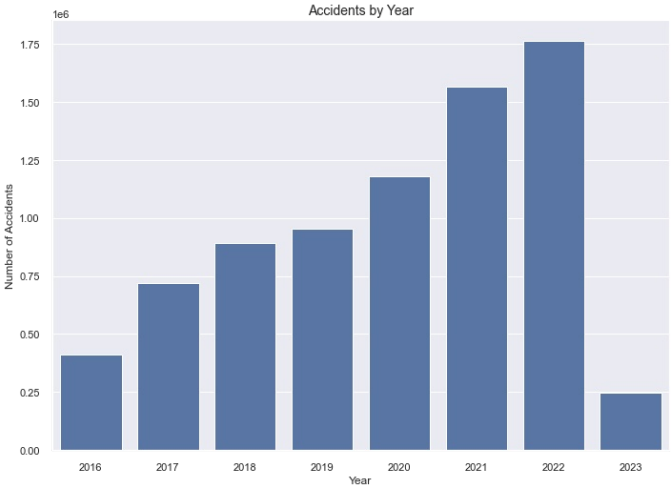
```
ax4.set_xlabel('Hour of Day', fontsize=12)
ax4.set_ylabel('Number of Accidents', fontsize=12)

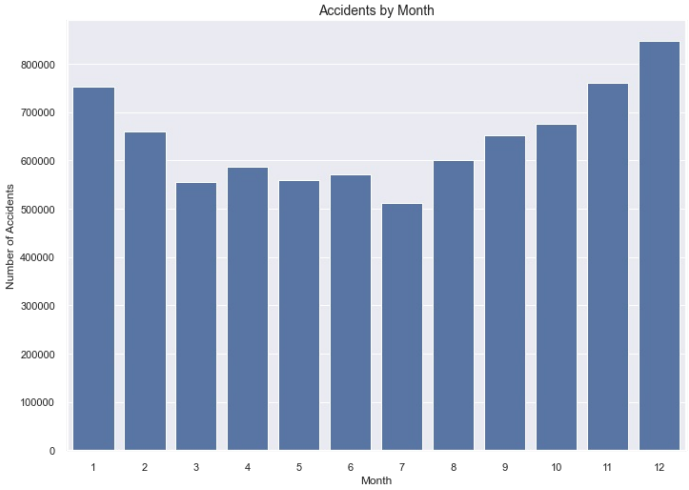
plt.tight_layout()
plt.show()

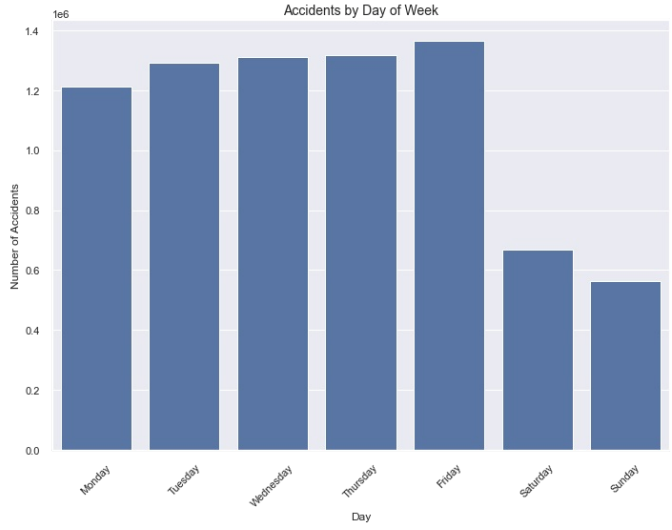
# Geographic distribution
plt.figure(figsize=(15, 10))
state_counts = df_clean['State'].value_counts()
sns.barplot(x=state_counts.index[:15], y=state_counts.values[:15])
plt.title('Top 15 States by Number of Accidents', fontsize=14)
plt.xlabel('State', fontsize=12)
plt.ylabel('Number of Accidents', fontsize=12)
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()

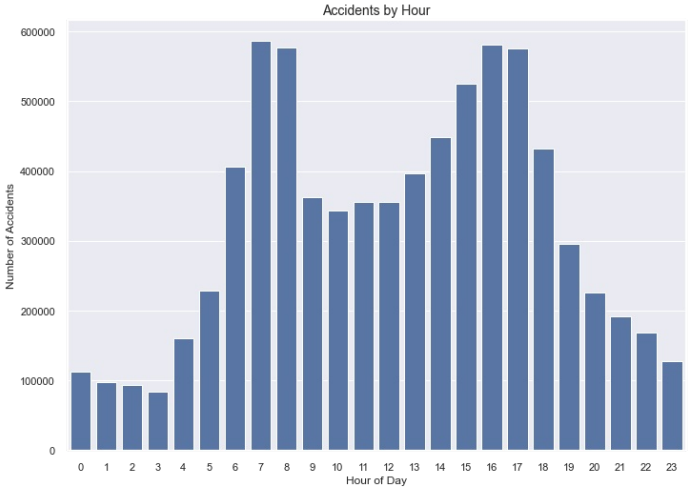
# Severity Analysis
plt.figure(figsize=(15, 5))
sns.countplot(data=df_clean, x='Severity')
plt.title('Distribution of Accident Severity', fontsize=14)
plt.xlabel('Severity Level', fontsize=12)
plt.ylabel('Number of Accidents', fontsize=12)
plt.show()

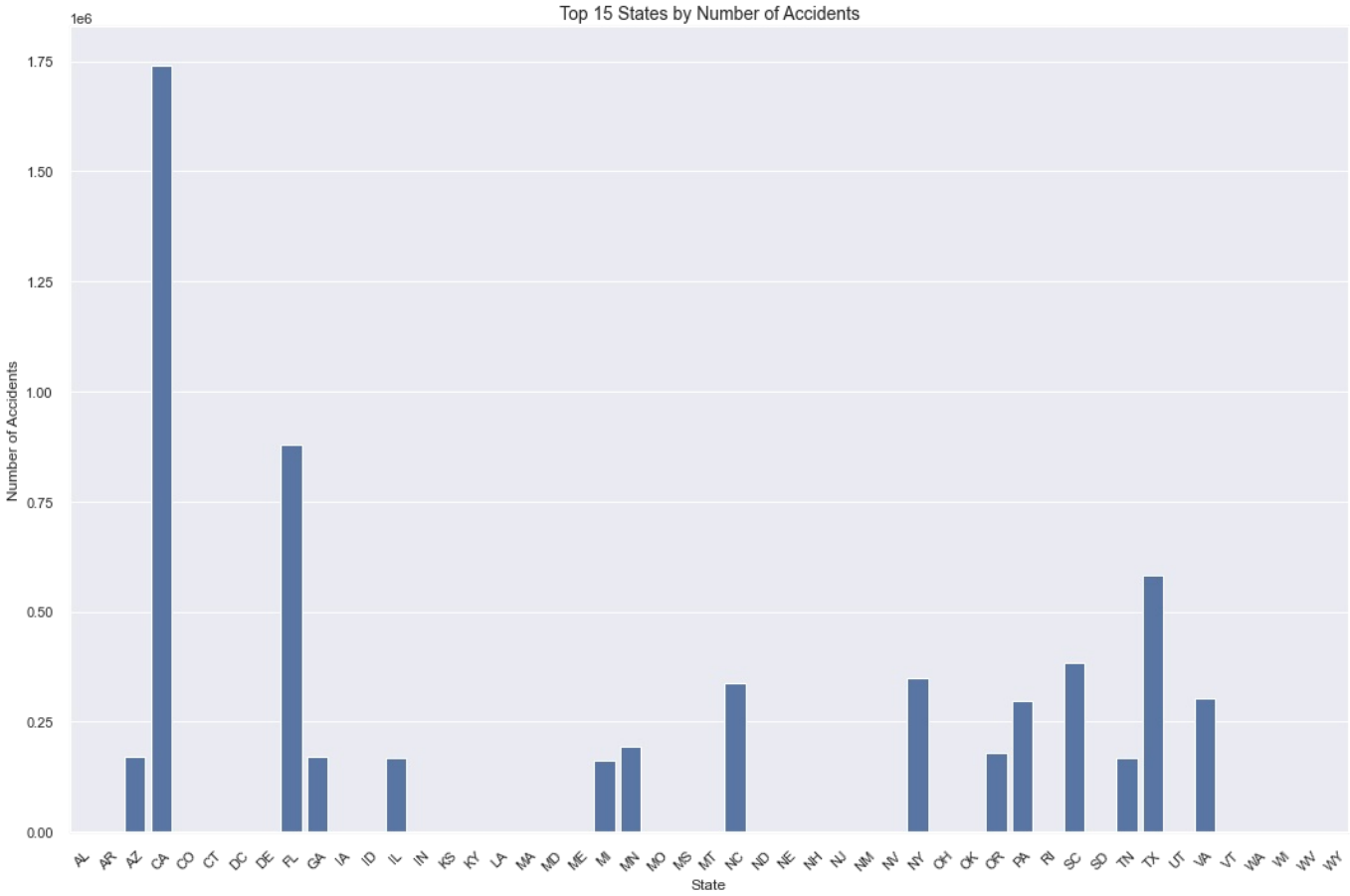
# Weather Impact
plt.figure(figsize=(15, 6))
weather_counts = df_clean['Weather_Condition'].value_counts().head(10)
sns.barplot(x=weather_counts.index, y=weather_counts.values)
plt.title('Top 10 Weather Conditions During Accidents', fontsize=14)
plt.xticks(rotation=45, ha='right')
plt.xlabel('Weather Condition', fontsize=12)
plt.ylabel('Number of Accidents', fontsize=12)
plt.tight_layout()
plt.show()
```

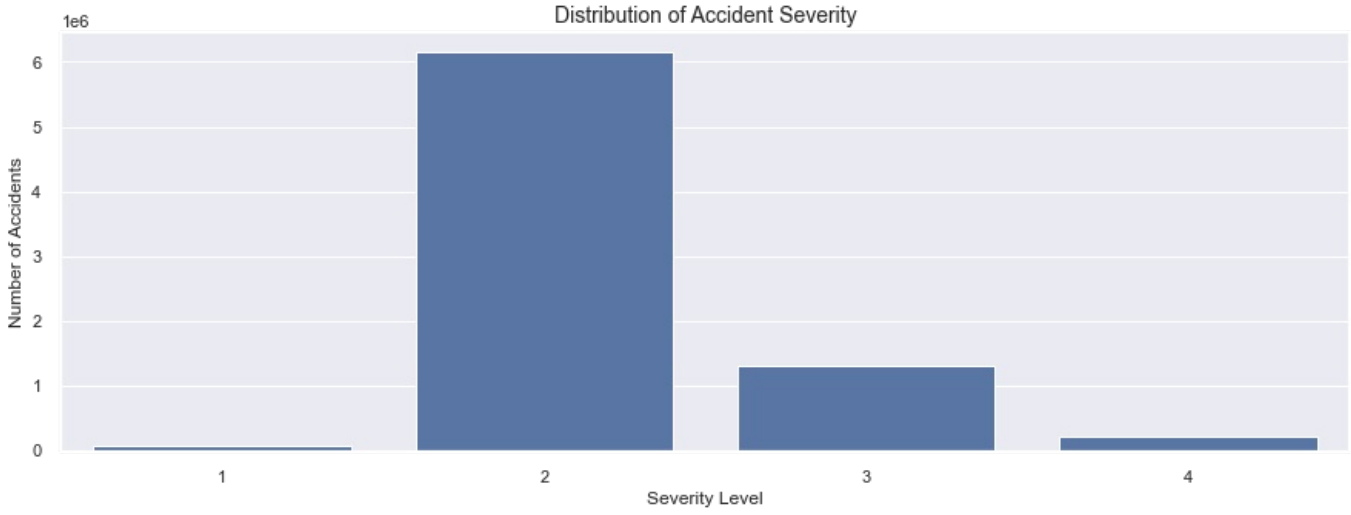


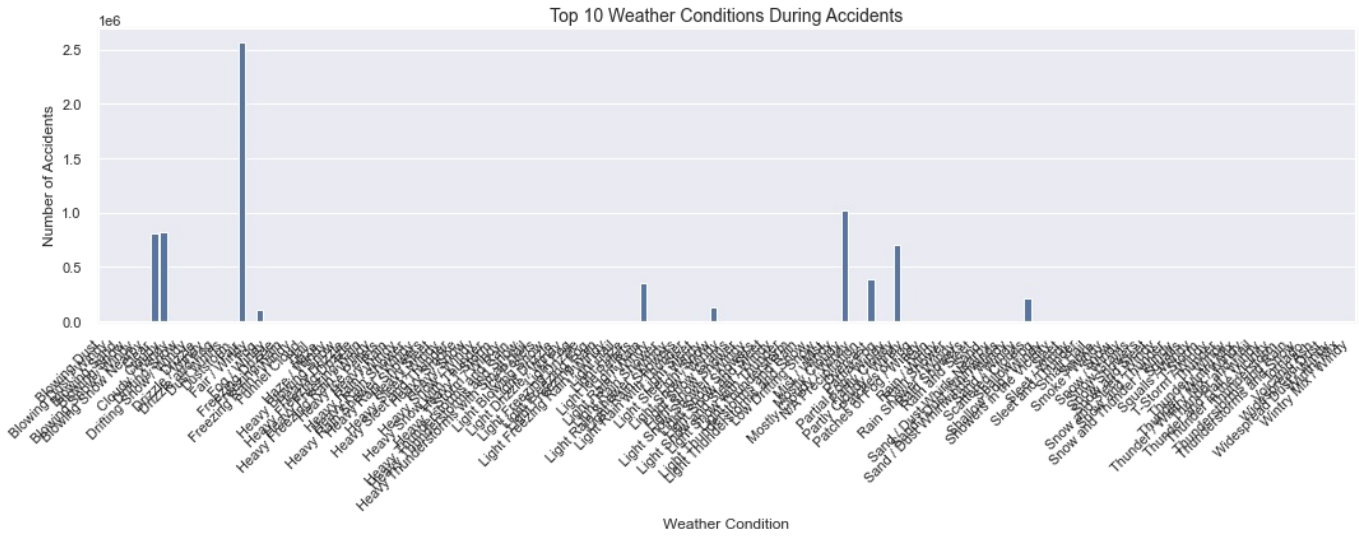







Top 15 States by Number of Accidents
1e6
1.75
1.50
1.25
1.00
0.75
0.50
0.25
0.00
Number of Accidents
AL AR AZ CA CO CT DC DE FL GA IA ID IL IN KS KY LA MA MD ME MI MN MO MS MT NC ND NE NH NJ NM NV NY OH OK OR PA RI SC SD TN TX UT VA VT WA WI WV WY
State


Distribution of Accident Severity
1e6
6
5
4
3
2
1
0
Number of Accidents
1
2
3
4
Severity Level



EDA Findings

Temporal Patterns

1. **Yearly Trends**

   > - Steady increase from 2016 (~0.4M) to 2022 (~1.75M) - Notable acceleration in growth between 2019-2021 - Sharp decline in 2023 data (partial year only)

2. **Monthly Distribution**

   > - December has highest frequency (~850K accidents) - July shows lowest frequency (~500K accidents) - Clear seasonal pattern with winter months showing 30-40% higher rates

3. **Weekly Pattern**

   > - Weekdays: ~1.2-1.3M each - Friday peak: ~1.35M - Weekend drop: Saturday (~0.65M) and Sunday (~0.55M) - ~50% reduction on weekends

4. **Daily Pattern**

   > - Peak periods: - Morning rush: 8AM (~580K) - Evening rush: 4-5PM (~570K) - Lowest: 3-4AM (~90K)

Geographic Distribution

> - California leads: ~1.7M accidents - Florida second: ~0.9M accidents - Most states: <0.3M accidents

Severity Analysis

- Level 2: ~6M accidents - Level 3: ~1.3M accidents - Level 4: ~0.2M accidents - Level 1: <0.1M accidents

Weather Conditions

- Visualization challenges: - Overcrowded labels - Similar condition distinction - Scale extremes - Key findings: - Fair weather dominates - Category consolidation needed - Low severe weather frequency

In [10]:
```
# Create improved weather impact visualizations
plt.style.use('seaborn-v0_8')
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(20, 8))

# Simplify weather conditions by grouping similar conditions
weather_mapping = {
    'Fair': 'Clear/Fair',
    'Clear': 'Clear/Fair',
    'Mostly Cloudy': 'Cloudy',
    'Partly Cloudy': 'Cloudy',
    'Cloudy': 'Cloudy',
    'Light Rain': 'Rain',
    'Rain': 'Rain',
    'Heavy Rain': 'Rain',
    'Snow': 'Snow',
    'Light Snow': 'Snow',
    'Heavy Snow': 'Snow',
    'Fog': 'Low Visibility',
    'Haze': 'Low Visibility',
    'Mist': 'Low Visibility'
}

# Create new column with simplified weather conditions
df_clean['Weather_Simple'] = df_clean['Weather_Condition'].map(lambda x: next((v for k, v in weather_mapping.it

# Plot 1: Simplified weather conditions distribution
weather_simple = df_clean['Weather_Simple'].value_counts()
total = len(df_clean)
weather_pct = (weather_simple / total * 100).round(1)

sns.barplot(x=weather_pct.values, y=weather_pct.index, ax=ax1, palette='viridis')
ax1.set_title('Weather Conditions Distribution', fontsize=14, pad=20)
ax1.set_xlabel('Percentage of Accidents (%)', fontsize=12)
ax1.set_ylabel('Weather Condition', fontsize=12)

# Add percentage labels
for i, v in enumerate(weather_pct):
    ax1.text(v + 0.5, i, f'{v}%', va='center', fontsize=10)

# Plot 2: Average severity by weather condition
severity_by_weather = df_clean.groupby('Weather_Simple')['Severity'].mean().sort_values(ascending=True)

sns.barplot(x=severity_by_weather.values, y=severity_by_weather.index, ax=ax2, palette='rocket')
ax2.set_title('Average Accident Severity by Weather Condition', fontsize=14, pad=20)
ax2.set_xlabel('Average Severity', fontsize=12)
ax2.set_ylabel('Weather Condition', fontsize=12)

# Add average severity labels
for i, v in enumerate(severity_by_weather):
    ax2.text(v + 0.01, i, f'{v:.2f}', va='center', fontsize=10)

plt.tight_layout()
plt.show()
```

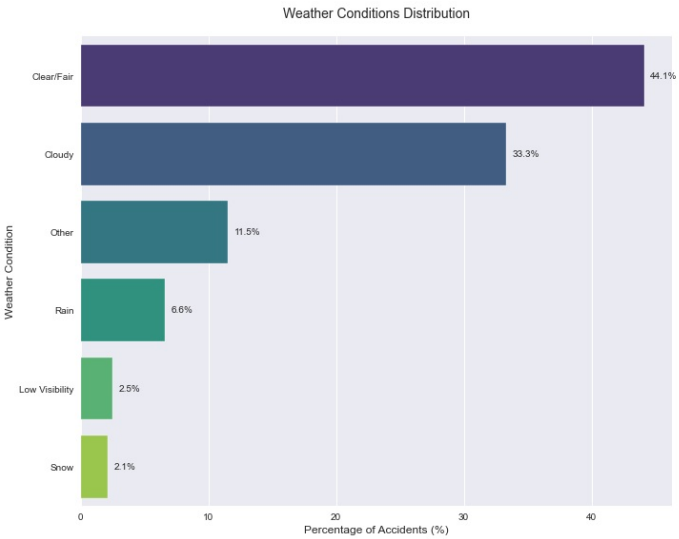
```
/var/folders/cv/87ynnq496l957k4sxtc2dshj99rmkw/T/ipykernel_15373/4036493381.py:31: FutureWarning:

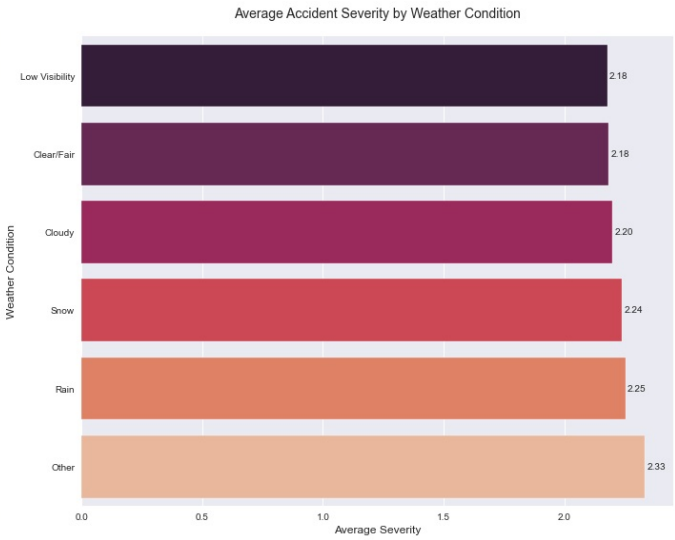
Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `y` variable
to `hue` and set `legend=False` for the same effect.

  sns.barplot(x=weather_pct.values, y=weather_pct.index, ax=ax1, palette='viridis')
/var/folders/cv/87ynnq496l957k4sxtc2dshj99rmkw/T/ipykernel_15373/4036493381.py:43: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `y` variable
to `hue` and set `legend=False` for the same effect.

  sns.barplot(x=severity_by_weather.values, y=severity_by_weather.index, ax=ax2, palette='rocket')
```





Weather Impact Analysis

Weather Conditions Distribution

1. **Clear/Fair Weather Dominates**

   - 44.1% of all accidents - Most common weather condition - Nearly half of recorded incidents

2. **Cloudy Conditions**

   - 33.3% of accidents - Second most frequent condition - Combined with clear/fair: 77.4% of total

3. **Other Weather Categories**

   - "Other": 11.5% - Rain: 6.6% - Low Visibility: 2.5% - Snow: 2.1%

Severity Analysis by Weather

1. **Adverse Weather Shows Higher Severity**

   - "Other" conditions: 2.33 average severity - Rain: 2.25 average severity - Snow: 2.24 average severity

2. **Moderate Conditions**

   - Cloudy weather: 2.20 average severity

3. **Clear Weather Severity**

   - Clear/Fair and Low Visibility: 2.18 average severity

Key Insights

- Paradox: Most common conditions show lowest severity - Narrow severity range (2.18-2.33) - Normal weather: 77%+ of accidents - Adverse conditions: only ~11% of incidents

Geographic Hotspot Analysis

Objectives

> 1. Create interactive heatmap 2. Identify major clusters 3. Visualize severity distribution

Approach

1. **Data Sampling** - 100,000 records for performance
2. **Base Map Creation**
   - US-centered coordinates
3. **Visualization Layers**
   - Heatmap for density
   - Severity-coded markers
   - Hotspot popups

Expected Insights

> Major accident cluster locations
>
> Urban area correlations
>
> Severity geographic patterns
>
> Highway/intersection impact

In [12]:
```
# First, import the required plugin
import folium.plugins

# Sample subset of data for visualization performance
sample_size = 100000
df_sample = df_clean.sample(n=sample_size, random_state=42)

# Create base map centered on US
base_map = folium.Map(location=[39.8283, -98.5795],
                      zoom_start=4,
                      tiles='cartodbpositron')

# Add heatmap layer
locations = df_sample[['Start_Lat', 'Start_Lng']].values.tolist()
folium.plugins.HeatMap(locations, radius=8, blur=10).add_to(base_map)

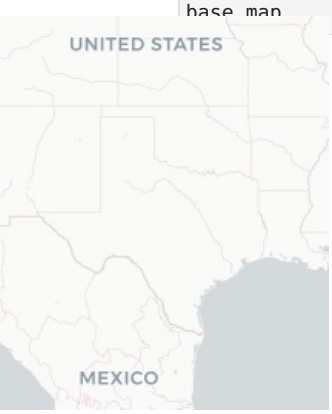
# Add circle markers colored by severity
def get_color(severity):
    if severity == 4:
        return 'red'
    elif severity == 3:
        return 'orange'
    elif severity == 2:
        return 'yellow'
    return 'green'

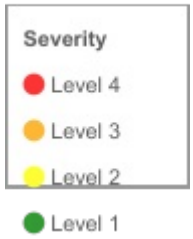
for idx, row in df_sample.iterrows():
    folium.CircleMarker(
        location=[row['Start_Lat'], row['Start_Lng']],
        radius=3,
        color=get_color(row['Severity']),
        popup=f"Severity: {row['Severity']}<br>"
              f"Weather: {row['Weather_Simple']}<br>"
              f"State: {row['State']}",
        fill=True
    ).add_to(base_map)

# Add legend
legend_html = '''
<div style="position: fixed;
            bottom: 50px; right: 50px; width: 120px; height: 120px;
            border:2px solid grey; z-index:9999; background-color:white;
            opacity:0.8;
            padding: 10px;
```

```
            font-size:14px;">
<p><b>Severity</b></p>
<p><i class="fa fa-circle" style="color:red"></i> Level 4</p>
<p><i class="fa fa-circle" style="color:orange"></i> Level 3</p>
<p><i class="fa fa-circle" style="color:yellow"></i> Level 2</p>
<p><i class="fa fa-circle" style="color:green"></i> Level 1</p>
</div>
'''
base_map.get_root().html.add_child(folium.Element(legend_html))

# Display map
base_map
```





Geographic Accident Distribution Analysis

Accident Hotspots Overview

1. **Major Urban Concentrations**

   > - Northeast Corridor: Boston-NYC-DC cluster - California: LA, SF Bay Area, Sacramento - Florida: Miami-Orlando-Tampa triangle - Texas: Houston, Dallas-Fort Worth, Austin

2. **Regional Patterns**

   > - Southeast: Major metro concentration - Midwest: Chicago-Detroit-Minneapolis corridor - West Coast: Interstate 5 corridor pattern - Mountain States: Denver and Salt Lake City focus

Severity Distribution

1. **Urban vs Rural**

   > - Level 2: Background pattern - Level 3: Suburban/interstate zones - Level 4: Metropolitan concentration

2. **Geographic Risk Factors**

   > - Interstate convergence: High severity clusters - Urban-rural interface: Elevated severity - Mountain passes: Scattered Level 3-4 incidents

Notable Features

- Population density correlation - Transportation corridor visibility - Coastal density patterns - Rural Level 2 consistency - Jurisdictional reporting gaps

Strategic Implications

- Emergency response allocation - Infrastructure improvement priorities - Urban planning considerations - Interstate safety measures

Temporal Pattern Analysis

Objectives

1. Time-series visualization creation 2. Multi-scale pattern analysis:
   - Hourly by weekday - Monthly by year - Seasonal variations
3. Severity-weather correlation 4. Peak period identification

Visualization Plan

- Hour vs Day Heatmap
- Monthly Trend Lines
- Severity Distribution Area Chart
- Weather-Time Analysis

In [13]:
```
# Set visualization style and figure size
plt.style.use('seaborn-v0_8')
fig = plt.figure(figsize=(20, 15))
gs = fig.add_gridspec(2, 2, hspace=0.3, wspace=0.3)

# 1. Heatmap: Hour vs Day of Week
ax1 = fig.add_subplot(gs[0, 0])
pivot_hour_day = pd.crosstab(df_clean['Hour'], df_clean['DayOfWeek'])
pivot_hour_day.columns = ['Mon', 'Tue', 'Wed', 'Thu', 'Fri', 'Sat', 'Sun']
sns.heatmap(pivot_hour_day, cmap='YlOrRd', ax=ax1)
ax1.set_title('Accident Frequency by Hour and Day', pad=20)
ax1.set_ylabel('Hour of Day')

# 2. Monthly trends by year
ax2 = fig.add_subplot(gs[0, 1])
monthly_yearly = df_clean.groupby(['Year', 'Month']).size().reset_index()
monthly_yearly.columns = ['Year', 'Month', 'Count']
for year in monthly_yearly['Year'].unique():
    year_data = monthly_yearly[monthly_yearly['Year'] == year]
    ax2.plot(year_data['Month'], year_data['Count'], label=str(year), marker='o')
ax2.set_xlabel('Month')
ax2.set_ylabel('Number of Accidents')
ax2.set_title('Monthly Accident Trends by Year')
ax2.legend(title='Year')
ax2.grid(True)

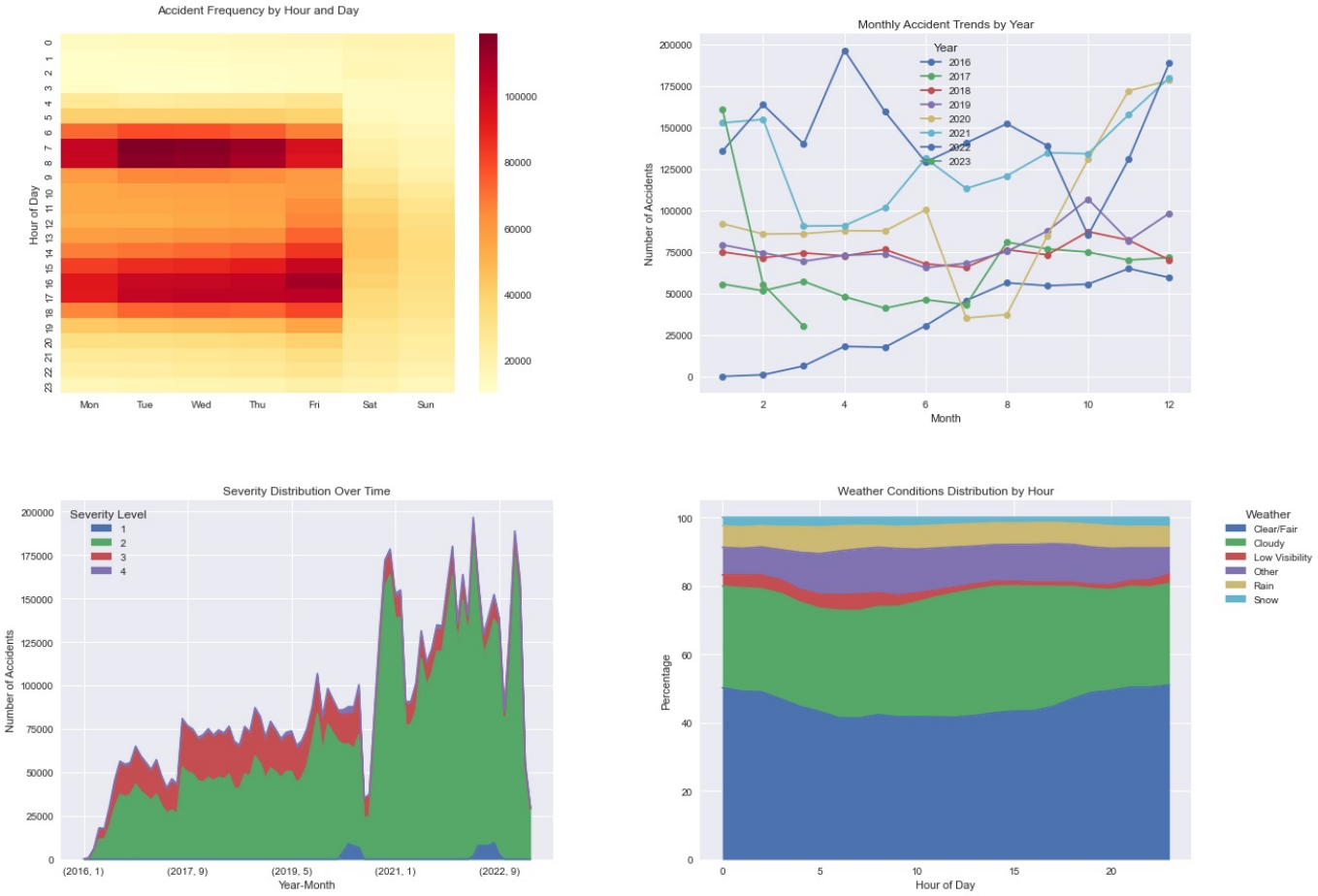
# 3. Severity distribution over time
ax3 = fig.add_subplot(gs[1, 0])
severity_time = df_clean.groupby(['Year', 'Month', 'Severity']).size().reset_index()
severity_pivot = severity_time.pivot_table(index=['Year', 'Month'],
                                           columns='Severity',
                                           values=0,
                                           fill_value=0)
severity_pivot.plot(kind='area', stacked=True, ax=ax3)
ax3.set_title('Severity Distribution Over Time')
ax3.set_xlabel('Year-Month')
ax3.set_ylabel('Number of Accidents')
ax3.legend(title='Severity Level')

# 4. Weather-Time correlation
```

```
ax4 = fig.add_subplot(gs[1, 1])
weather_hour = pd.crosstab(df_clean['Hour'], df_clean['Weather_Simple'])
weather_hour_pct = weather_hour.div(weather_hour.sum(axis=1), axis=0) * 100
weather_hour_pct.plot(kind='area', stacked=True, ax=ax4)
ax4.set_title('Weather Conditions Distribution by Hour')
ax4.set_xlabel('Hour of Day')
ax4.set_ylabel('Percentage')
ax4.legend(title='Weather', bbox_to_anchor=(1.05, 1))

plt.tight_layout()
plt.show()
```

```
/var/folders/cv/87ynnq496l957k4sxtc2dshj99rmkw/T/ipykernel_15373/1395626620.py:50: UserWarning: This figure incl
udes Axes that are not compatible with tight_layout, so results might be incorrect.
  plt.tight_layout()
```



Temporal Analysis of Traffic Accidents

Hour-Day Heatmap Analysis

1. **Peak Hours**

> - Morning Rush (7-9 AM): High intensity - Evening Rush (3-7 PM): Maximum frequency - Weekday consistency - Weekend reduction

2. **Off-Peak Patterns**

> - 11 PM-4 AM: Minimal activity - 10 AM-2 PM: Moderate levels - Weekends: Reduced intensity

Monthly Trends by Year

1. **Annual Progression**

> - 2016-2021: Upward trend - December: Consistent peak - Summer: Stable patterns

2. **Year-Specific Patterns**

- 2020-2021: Frequency spike - 2016-2019: Pattern stability - Recent: Monthly volatility

Severity Distribution

- Level 2 (green): Predominant - 2016-2022: Significant growth - Level 3-4: Consistent ratio - Level 1: Minimal presence - Winter: Severity spikes

Weather Conditions by Hour

1. **Daily Pattern**

   - Clear/Fair: Dominant - Mid-day: Increased cloudiness - Rain/snow: Stable distribution - Early morning: Visibility issues

2. **Distribution Features**

   - Weather: Daily stability - Clear/cloudy: Minor variations - Adverse conditions: Consistent percentage - "Other": Minimal fluctuation

Weather Impact Analysis with Statistical Methods

Analytical Approach

1. **Statistical Tests**

   - Chi-square: Weather-severity relationship - ANOVA: Weather impact on duration - Correlation: Weather metrics analysis

2. **Machine Learning**

   - Feature importance analysis - Weather condition clustering - Severity prediction model

3. **Key Weather Features**

   - Temperature
   - Precipitation
   - Wind Speed
   - Visibility
   - Weather_Simple

In [15]:
```
import scipy.stats as stats
from sklearn.preprocessing import StandardScaler
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report
import warnings
warnings.filterwarnings('ignore')

# 1. Statistical Analysis
# Chi-square test for weather-severity relationship
contingency = pd.crosstab(df_clean['Weather_Simple'], df_clean['Severity'])
# Unpack all 4 values from chi2_contingency
chi2, p_value, dof, expected = stats.chi2_contingency(contingency)

# ANOVA for weather impact on duration
weather_groups = [group for _, group in df_clean.groupby('Weather_Simple')['Duration']]
f_stat, anova_p = stats.f_oneway(*weather_groups)
```

```python
# 2. Create weather feature matrix
weather_features = ['Temperature(F)', 'Wind_Speed(mph)', 'Visibility(mi)', 'Pressure(in)']
X = df_clean[weather_features].copy()
y = df_clean['Severity']

# Handle missing values
X = X.fillna(X.mean())

# Scale features
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
X_scaled = pd.DataFrame(X_scaled, columns=weather_features)

# 3. Train Random Forest for feature importance
X_train, X_test, y_train, y_test = train_test_split(X_scaled, y, test_size=0.2, random_state=42)
rf = RandomForestClassifier(n_estimators=100, random_state=42)
rf.fit(X_train, y_train)

# Create visualizations
fig, ((ax1, ax2), (ax3, ax4)) = plt.subplots(2, 2, figsize=(20, 15))

# 1. Weather-Severity Heatmap
severity_weather_norm = contingency.div(contingency.sum(axis=1), axis=0)
sns.heatmap(severity_weather_norm, annot=True, fmt='.2%', cmap='YlOrRd', ax=ax1)
ax1.set_title('Weather Condition vs Severity (Normalized)')
ax1.set_xlabel('Severity Level')
ax1.set_ylabel('Weather Condition')

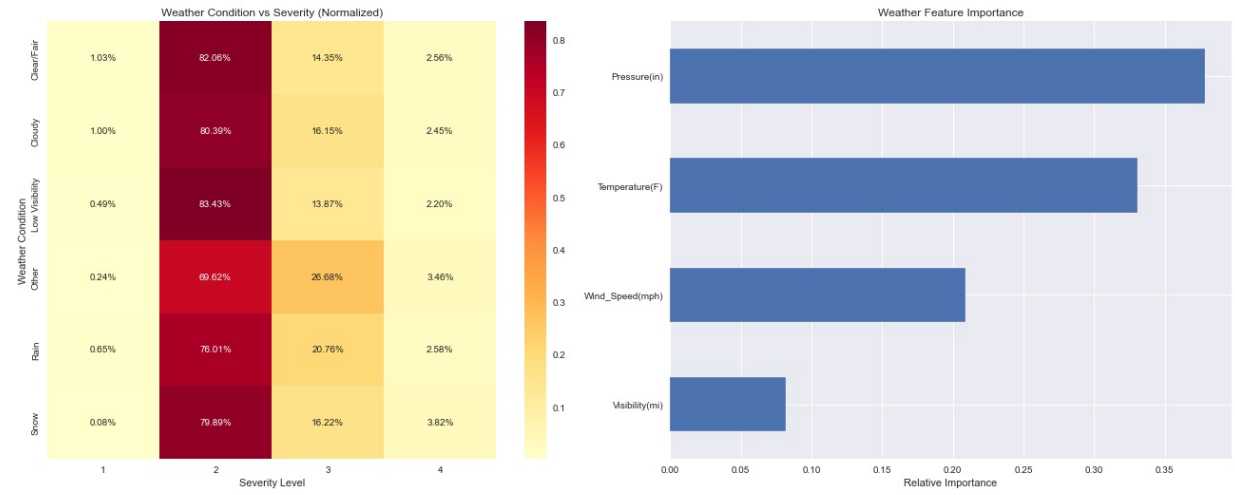
# 2. Feature Importance Plot
importances = pd.Series(rf.feature_importances_, index=weather_features)
importances.sort_values().plot(kind='barh', ax=ax2)
ax2.set_title('Weather Feature Importance')
ax2.set_xlabel('Relative Importance')

# 3. Weather Conditions vs Duration
sns.boxplot(data=df_clean, x='Weather_Simple', y='Duration', ax=ax3)
ax3.set_xticklabels(ax3.get_xticklabels(), rotation=45)
ax3.set_title('Accident Duration by Weather Condition')
ax3.set_xlabel('Weather Condition')
ax3.set_ylabel('Duration (Hours)')

# 4. Temperature vs Severity with Weather
sns.scatterplot(data=df_clean, x='Temperature(F)', y='Severity',
                hue='Weather_Simple', alpha=0.5, ax=ax4)
ax4.set_title('Temperature vs Severity by Weather Condition')
ax4.legend(bbox_to_anchor=(1.05, 1))

plt.tight_layout()
plt.show()

# Print statistical results
print(f"\nChi-square test statistics:")
print(f"Chi-square value: {chi2:.2f}")
print(f"p-value: {p_value:.2e}")
print(f"Degrees of freedom: {dof}")
print(f"\nANOVA test p-value: {anova_p:.2e}")
print("\nRandom Forest Classification Report:")
print(classification_report(y_test, rf.predict(X_test)))
```




```
Chi-square test statistics:
Chi-square value: 96229.39
p-value: 0.00e+00
Degrees of freedom: 15

ANOVA test p-value: 5.24e-49

Random Forest Classification Report:
              precision    recall  f1-score   support

           1       0.40      0.07      0.12     13677
           2       0.81      0.96      0.88   1230834
           3       0.39      0.12      0.19    260285
           4       0.36      0.08      0.13     40833

    accuracy                           0.78   1545629
   macro avg       0.49      0.31      0.33   1545629
weighted avg       0.72      0.78      0.73   1545629
```

Weather Impact on Accident Severity Analysis

Statistical Significance

1. **Chi-Square Results**

> - Value: 96229.39 - P-value: 0.00e+00 - Degrees of freedom: 15

2. **ANOVA Results**

> - P-value: 5.24e-49 - Strong weather-severity relationship

Feature Importance

- Pressure: ~0.35 - Temperature: ~0.33 - Wind Speed: ~0.22 - Visibility: ~0.10

Severity Distribution

- Level 2: 76-83% - Level 3: 13-26% - Level 1: 0.24-1.03% - Level 4: 2.2-3.8%

Condition Patterns

- Snow: Highest Level 4 (3.82%) - "Other": Highest Level 3 (26.68%) - Clear/Fair: Most balanced

Model Performance

1. **Overall Metrics** - Accuracy: 0.78 - F1-score: 0.73

2. **Class Performance**
   - Severity 2: F1 0.88
   - Severity 1: Recall 0.07
   - Severity 3 & 4: Limited success

Duration Analysis

- Most cases: <10,000 hours - Extremes: >30,000 hours - Clear/Fair: Highest frequency - Consistent outliers across conditions

Conclusion and Recommendations

Key Findings

1. **Statistical Patterns**

   - Weather-severity correlation: χ^2 = 96229.39 - Urban corridor clustering - Pressure (35%) and temperature (33%) dominance - Regional weather patterns

2. **Environmental Interactions**

   - Regional pressure variations - Geographic temperature effects - Urban heat island impact - Coastal patterns

3. **Impact Analysis**

   - Level 2 severity: 76-83% - Population density correlation - Regional duration variations - Interstate patterns

Implementation

1. **Stakeholder Tools**

   - GIS visualization - Traffic management dashboards - Public mapping - Data sharing protocols

2. **Education**

   - Spatial data workshops - GIS documentation - Tool training - Interactive tutorials

3. **Infrastructure**

   - Resource allocation maps - Predictive modeling - Risk assessment - Weather-responsive systems

Future Development

1. **Technical** - ML-GIS integration - Automated pattern recognition - Real-time visualization - Classification improvement
2. **Data**
   - Standardized collection
   - Metadata standards
   - Unified architecture
   - Cross-jurisdictional sharing
3. **Collaboration**
   - Transportation authorities
   - Academic research
   - Emergency services
   - Community reporting

In [16]:
```python
def export_analysis_results(df_clean, rf_model, chi2_stats, analysis_path="analysis_results"):
    """
    Export analysis results, visualizations, and statistical findings
    """
    import os
    import json
    from datetime import datetime

    # Create analysis directory
    os.makedirs(analysis_path, exist_ok=True)
    timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")

    # Export key statistics
    stats = {
        "chi_square": {
            "value": float(chi2_stats[0]),
            "p_value": float(chi2_stats[1]),
            "dof": int(chi2_stats[2])
        },
        "feature_importance": {
            feature: float(importance)
            for feature, importance in zip(
                ['Temperature(F)', 'Wind_Speed(mph)', 'Visibility(mi)', 'Pressure(in)'],
                rf_model.feature_importances_
            )
        },
        "data_summary": {
            "total_accidents": len(df_clean),
            "date_range": f"{df_clean['Start_Time'].min()} to {df_clean['Start_Time'].max()}",
            "states_covered": len(df_clean['State'].unique())
        }
    }

    # Save statistics
    with open(f"{analysis_path}/statistics_{timestamp}.json", "w") as f:
        json.dump(stats, f, indent=4)

    # Save aggregated data
    severity_by_state = df_clean.groupby(['State', 'Severity']).size().unstack()
    severity_by_state.to_csv(f"{analysis_path}/severity_by_state_{timestamp}.csv")

    print(f"Analysis results exported to {analysis_path}/")

# Call function with our analysis results
export_analysis_results(df_clean, rf, [chi2, p_value, dof])
```

```
Analysis results exported to analysis_results/
```

Loading [MathJax]/extensions/Safe.js